Crear un juego de plataformas con GlasnostFWK

Tutorial mantenible, paso a paso, basado en el flujo de construcción y en las APIs presentes en el repositorio.

Edición: 31 de octubre de 2026 · Perfil estable `standard-160x200.json` · Ensamblador Z80

> **Alcance confirmado.** Esta guía explica cómo montar un proyecto pequeño con movimiento, salto, tilemap, colisión, sprites y objetos coleccionables sobre `video/entity_render`. No presupone cámara con scroll, pendientes, streaming de mapas ni asignación dinámica de entidades: esas funciones no forman parte del contrato actual.

Índice

1. Resultado y requisitos
2. Crear la estructura
3. Declarar el manifiesto
4. Preparar mapa y assets
5. Definir jugador y física
6. Inicializar el juego
7. Construir el bucle
8. Añadir objetos y HUD
9. Construir y ejecutar
10. Integrar en el DSK común
11. Pruebas y diagnóstico
12. Regenerar esta documentación

1. Resultado y requisitos

El proyecto queda dividido en cuatro archivos: `example.json` elige perfil y módulos, `game.inc` comparte constantes, `assets.s` conserva datos estáticos, y `main.s` implementa estado, inicialización y bucle. El constructor prepara un proyecto CPCTelera aislado bajo `.build/<nombre>`; no copie ni edite a mano ese directorio generado.

Antes de empezar, desde la raíz de GlasnostFWK, compruebe:

```
test -f /config/workspace/cpctelera_next/cpctelera/cpctelera.lib
python3 --version
make --version
```

Si CPCTelera está en otra ruta, defina `CPCT_NEXT_PATH` al invocar el constructor. El valor predeterminado del repositorio es `/config/workspace/cpctelera_next/cpctelera`.

> **Unidades.** En el perfil estándar, X y el ancho usados por sprites y por este perfil de plataformas están expresados en bytes Mode 0; cada byte contiene dos píxeles. Y y la altura se expresan en scanlines. Mezclar píxeles con bytes desplaza colisiones y dibujos.



Captura 1. Escena actual obtenida con AmSpiriT-Lite 1.15.3, modelo CPC 6128, después de que el smoke validara la inicialización del juego con tiles de 4x8 bytes.

2 Crear la estructura

Para seguir el tutorial sin modificar el ejemplo distribuido, utilice otro nombre:

```
mkdir -p examples/mi_plataformas
touch examples/mi_plataformas/example.json
touch examples/mi_plataformas/main.s
touch examples/mi_plataformas/assets.s
touch examples/mi_plataformas/game.inc
```

La captura siguiente se genera leyendo el ejemplo real. Muestra la misma separación que debe conservar el proyecto nuevo y los módulos de plataforma disponibles.

Estructura real del ejemplo

```
$ find examples/platformer -maxdepth 1 -type f -printf '%f\n' | sort
assets.s
example.json
game.inc
main.s

$ find src/extensions -maxdepth 1 -name '*platform*' -printf '%f\n' | sort
physics_platform.inc
physics_platform.s
platform_tilemap.inc
platform_tilemap.s
```

Captura 2. Estructura obtenida mediante `find`; no es un esquema dibujado a mano.

Archivo	Responsabilidad recomendada
`example.json`	Dependencias explícitas, perfil de vídeo, nombre AMSDOS y fuentes del juego.
`game.inc`	Offsets, máscaras y constantes compartidas entre código y assets.
`assets.s`	Mapa row-major, tabla de tipos, tileset, sprites, configuración, teclas y posiciones iniciales.
`main.s`	Estado mutable, inicialización, lectura de entrada, física, reglas y render.

3 Declarar el manifiesto

Escriba en `examples/mi_plataformas/example.json`:

```
{
  "name": "mi_plataformas",
  "amsdos_name": "MIPLATA",
  "profile": "standard-160x200.json",
  "transparency": {"pen": 0},
  "modules": [
    "core/system.s",
    "core/input.s",
    "video/video.s",
    "video/sprite.s",
    "video/entity_render.s",
    "ui/text.s",
    "extensions/physics_platform.s",
    "extensions/tilemap.s",
    "extensions/tilemap_render.s",
    "extensions/platform_tilemap.s"
  ],
  "game_sources": ["main.s", "assets.s", "game.inc"]
}
```

`game_sources` debe incluir `main.s`. `transparency.pen=0` habilita el renderer transparente y coincide con el fondo de la fuente incorporada. Las rutas son relativas al directorio del ejemplo y el constructor rechaza rutas que escapen de él. `amsdos_name` se usa al integrar el BIN en el DSK común: debe contener entre uno y ocho caracteres AMSDOS válidos.

Manifiesto real del ejemplo

```
$ sed -n '1,120p' examples/platformer/example.json
{
  "name": "platformer",
  "amsdos_name": "PLATGAME",
  "profile": "standard-160x200.json",
  "transparency": {
    "pen": 0
  },
  "modules": [
    "core/system.s",
    "core/input.s",
    "video/video.s",
    "video/sprite.s",
    "video/entity_render.s",
    "ui/text.s",
    "extensions/physics_platform.s",
    "extensions/tilemap.s",
    "extensions/tilemap_render.s",
    "extensions/platform_tilemap.s"
  ],
  "game_sources": [
    "main.s",
    "assets.s",
    "game.inc"
  ]
}
```

Captura 3. Manifiesto leído directamente de `examples/platformer/example.json`. Si el ejemplo evoluciona, el generador de capturas reflejará el cambio.

> **Dependencias explícitas.** Añadir una llamada en ensamblador no enlaza automáticamente su implementación. El archivo que define la rutina debe aparecer en `modules`; los `.inc` asociados se copian automáticamente cuando existen.

3.1 Compartir constantes del juego

El constructor copia las fuentes del consumidor bajo `src/game`. Por eso `main.s` y `assets.s` incluyen el archivo compartido con esta ruta:

```
.include "game/game.inc"
```

Centralice allí offsets y reglas numéricas para no duplicarlas:

```
PF_PLAYER_X      = 0
PF_PLAYER_Y      = 1
PF_PLAYER_VX     = 2
PF_PLAYER_VY     = 3
PF_PLAYER_WIDTH  = 4
PF_PLAYER_HEIGHT = 5
PF_PLAYER_FLAGS  = 6
PF_PLAYER_SIZE   = 7

PF_INPUT_LEFT    = 0x01
PF_INPUT_RIGHT   = 0x02
PF_INPUT_JUMP    = 0x04
PF_SPEED         = 2
PF_JUMP_SPEED    = -8
```

La ruta preparada mantiene una actualización por frame en el tramo diagnosticado con monedas visibles, por lo que `PF_SPEED=2` expresa directamente el ritmo horizontal original. No es una garantía general de rendimiento: mida de nuevo si cambia el número, tamaño o tipo de entidades.

4 Preparar mapa, tiles y sprites

4.1 Mapa y tipos de colisión

El mapa contiene identificadores de tile en orden de filas. Otra tabla traduce cada identificador a `FWK_TILE_PASSABLE`, `FWK_TILE_SOLID` o `FWK_TILE_ONE_WAY`. El ejemplo usa las rutinas nativas de CPCTelera Next con tiles de 4 bytes por 8 scanlines: 8x8 píxeles en Mode 0 y 20 columnas para cubrir la pantalla.

```
.include "cpctelera.h.s"
.include "extensions/tilemap.inc"
.include "extensions/tilemap_render.inc"
.include "video/entity_render.inc"

tile_types: .db FWK_TILE_PASSABLE, FWK_TILE_SOLID, FWK_TILE_ONE_WAY
level_map:
    .db 1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1
    .db 1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1
    ; ...añada las filas restantes...
    .db 1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1

FWK_TILEMAP_DESCRIPTOR level_descriptor, level_map, tile_types, 20, 25, 2, 3, 4,
8, 3
```

Los tamaños deben ser coherentes con los desplazamientos: 4 bytes implica shift X 2 y 8 scanlines implica shift Y 3. El render opaco solo acepta las geometrías CPCTelera 2x4, 2x8, 4x4 y 4x8 bytes. Las consultas fuera del mapa y los identificadores inválidos se tratan como error/transitable, no como una pared implícita.

4.2 Tileset y vista

Cada gráfico ocupa exactamente `x_size * y_size` bytes Mode 0, por filas y sin padding. Concatenelos por identificador y describa la región visible:

```
tileset:
    .rept 32
        .db 0x00        ; tile 0: 4 bytes x 8 scanlines
    .endm
    .rept 32
        .db 0x55        ; tile 1
    .endm
    .rept 32
        .db 0xF0        ; tile 2
    .endm

FWK_TILE_RENDER_DESCRIPTOR level_view, tileset, 0, 0, 0, 0, 20, 25, 3
```

La cámara y el viewport están en tiles; `screen_x` está en bytes Mode 0 y `screen_y` es una palabra alineada al alto del tile. El render completo valida la geometría y llama a `cpct_drawTileAligned4x8_f_asm` para cada celda.

4.3 Prefijo de render y buffer de fondo

```
player_pixels:
    .rept 8
        .db 0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF
    .endm

player_background: .ds 8 * 8
player_entity:
    FWK_ENTITY_RENDER_PREFIX player_pixels, 16, 184, 8, 8, 192, player_background,
64, FWK_ENTITY_RENDER_VISIBLE, 0, 0
```

El prefijo ocupa 25 bytes y debe comenzar en el offset cero de la entidad. Contiene el descriptor de sprite compatible, `sort_y`, el buffer de fondo, su capacidad, la geometría anterior, flags y pens de recoloreado. Reserve como mínimo `width * height` bytes para cada entidad que pueda estar activa.

5 Definir jugador, física y controles

El perfil compacto usa un estado de siete bytes: `x, y, vx, vy, width, height, flags`. En este ejemplo comienza justo después del prefijo de render; así las rutinas de física reciben el subregistro compacto mediante `IY`, mientras el renderer recibe la base de la entidad mediante su array de punteros.

```
.include "extensions/physics_platform.inc"

player_background: .ds 8 * 8
player_entity:
    FWK_ENTITY_RENDER_PREFIX player_pixels, 16, 184, 8, 8, 192, player_background,
64, FWK_ENTITY_RENDER_VISIBLE, 0, 0
player_state: .ds FWK_PLATFORM_STATE_SIZE

entity_ptrs: .dw player_entity, coin0_entity, coin1_entity
render_work: .ds 3 * FWK_ENTITY_RENDER_WORK_SIZE
FWK_ENTITY_RENDER_DESCRIPTOR render_list, entity_ptrs, 3, render_work, 3

FWK_PLATFORM_CONFIG physics_config, 80, 255, 1, 8

ACTION_LEFT  = 0x01
ACTION_RIGHT = 0x02
ACTION_JUMP  = 0x04

bindings:
    .dw Key_CursorLeft
    .db ACTION_LEFT
    .dw Key_CursorRight
    .db ACTION_RIGHT
    .dw Key_CursorUp
    .db ACTION_JUMP
    .dw 0
    .db 0
```

La máscara cero termina la tabla. Hay como máximo ocho bits de acción. Use `fwk_input_held` para movimiento continuo y `fwk_input_pressed` para disparar el salto una sola vez.

El suelo global en 255 actúa solo como límite de seguridad si el tilemap ya posee un suelo visible. El resolver de mapa establece `FWK_PLATFORM_TILE_GROUNDED` al aterrizar sobre tiles sólidos o plataformas de una dirección.

6 Inicializar el juego

La secuencia mínima es: configuración del proyecto, vídeo, módulos auxiliares, estado, entrada y escena estática. `fwk_project_init` inicializa el sistema e instala una sola vez la tabla de transparencia configurada.

```
_main::
    call fwk_project_init
    call fwk_video_init
    call fwk_text_init
    xor a
    call fwk_video_clear_visible

    ld a, #16
    ld (player_state + FWK_PLATFORM_X), a
    ld a, #184
    ld (player_state + FWK_PLATFORM_Y), a
    xor a
    ld (player_state + FWK_PLATFORM_VX), a
    ld (player_state + FWK_PLATFORM_VY), a
    ld a, #8
    ld (player_state + FWK_PLATFORM_WIDTH), a
    ld (player_state + FWK_PLATFORM_HEIGHT), a
    ld a, #FWK_PLATFORM_TILE_GROUNDED
    ld (player_state + FWK_PLATFORM_FLAGS), a

    call fwk_input_init
    ld hl, #bindings
    call fwk_input_set_bindings

    ld ix, #level_descriptor
    ld iy, #level_view
    call fwk_tilemap_render_view
    jp c, fatal_data_error

    ld ix, #render_list
    call fwk_entity_render_init
    jp c, fatal_data_error
    call fwk_entity_render_prepare_dirty
    call restore_entities
    call draw_entities
```

`fwk_entity_render_init` valida la lista y marca las entidades visibles para el primer dibujo. Si devuelve carry, revise punteros duplicados, capacidad del array auxiliar, flags, geometrías, spans y buffers.

7 Construir el bucle de juego

El orden respeta la máquina de estados `IDLE -> RESTORED -> IDLE` y garantiza que el resolver pueda reconstruir la posición anterior a partir de la velocidad:

1. Esperar el frame.
2. Actualizar entrada y velocidades.
3. Ejecutar física.
4. Resolver inmediatamente la colisión con el tilemap.
5. Aplicar reglas del juego.
6. Sincronizar posición y `sort_y` del prefijo; activar `DIRTY` si cambió.
7. Marcar entidades afectadas por cambios externos del fondo.
8. Restaurar todas las entidades sucias, redibujar el fondo externo y dibujar las entidades.

```
game_loop:
    call fwk_input_update
    call update_horizontal_velocity
    call try_jump

    ld ix, #physics_config
    ld iy, #player_state
    call fwk_physics_platform_step

    ld ix, #level_descriptor
    ld iy, #player_state
    call fwk_platform_tilemap_resolve
    ; A contiene la mascara FWK_PLATFORM_HIT_*

    call update_rules
    call sync_player_render

    ; Prepare validacion, cierre y orden sin tocar VRAM.
    ld ix, #render_list
    call fwk_entity_render_prepare_dirty
    jp c, fatal_render_error
    ; Tras VSYNC no espere ni haga logica entre restauracion y dibujo.
    call fwk_system_wait_frame
    ld ix, #render_list
    call fwk_entity_render_restore_dirty
    jp c, fatal_render_error
    ld ix, #render_list
    call fwk_entity_render_draw_dirty
    jp c, fatal_render_error
    jp game_loop
```

7.1 Movimiento horizontal

```
    ld a, (fwk_input_held)
    and #ACTION_LEFT
    jr z, check_right
    ld a, #-4
    ld (player_state + FWK_PLATFORM_VX), a
    jr horizontal_done
check_right:
    ld a, (fwk_input_held)
    and #ACTION_RIGHT
    jr z, stop_horizontal
    ld a, #4
    ld (player_state + FWK_PLATFORM_VX), a
    jr horizontal_done
stop_horizontal:
    xor a
    ld (player_state + FWK_PLATFORM_VX), a
horizontal_done:
```

7.2 Salto

```
    ld a, (fwk_input_pressed)
    and #ACTION_JUMP
    jr z, no_jump
    ld a, (player_state + FWK_PLATFORM_FLAGS)
    and #FWK_PLATFORM_TILE_GROUNDED
    jr z, no_jump
    ld a, #-8
    ld (player_state + FWK_PLATFORM_VY), a
no_jump:
```

La velocidad es un byte con signo. La rutina de física aplica gravedad y caída máxima; el resolver corrige paredes, techo y suelo. No use velocidades que permitan atravesar un tile completo en un solo frame: no hay barrido continuo ni subdivisión automática.

7.3 Sincronizar el jugador con el renderer

```
sync_player_render:
    ; Marque DIRTY solo si la posicion compacta ha cambiado.
    ld a, (player_state + FWK_PLATFORM_X)
    ld b, a
    ld a, (player_entity + FWK_ENTITY_RENDER_X)
    cp b
    jr nz, player_changed
    ld a, (player_state + FWK_PLATFORM_Y)
    ld b, a
    ld a, (player_entity + FWK_ENTITY_RENDER_Y)
    cp b
    jr z, player_store
player_changed:
    ld hl, #player_entity + FWK_ENTITY_RENDER_FLAGS
    set 1, (hl) ; FWK_ENTITY_RENDER_DIRTY
player_store:
    ld a, (player_state + FWK_PLATFORM_X)
    ld (player_entity + FWK_ENTITY_RENDER_X), a
    xor a
    ld (player_entity + FWK_ENTITY_RENDER_X + 1), a
    ld a, (player_state + FWK_PLATFORM_Y)
    ld (player_entity + FWK_ENTITY_RENDER_Y), a
    ld l, a
    ld h, #0
    xor a
    ld (player_entity + FWK_ENTITY_RENDER_Y + 1), a
    ld de, #8
    add hl, de
    ld (player_entity + FWK_ENTITY_RENDER_SORT_Y), hl
    ret
```

> **Detalle importante.** No cambie entidades desde `prepare_dirty` hasta `draw_dirty`. Si modifica punteros o rangos de buffer en `IDLE`, llame a `revalidate`. El descriptor almacena X e Y en 16 bits con signo, mientras el estado compacto usa bytes; inicialice también el byte alto. `sort_y` es la clave estable de orden, no una coordenada usada por la física.

8 Añadir objetos coleccionables y HUD

Una regla sencilla almacena cada objeto como `x_tile, y_tile, tipo, puntero_entidad`. En cada frame, compare su rectángulo con el del jugador. Al recogerlo:

1. marque la entrada lógica como inactiva;
2. quite `VISIBLE` y active `DIRTY` en su entidad;
3. incremente la puntuación;
4. marque el HUD como sucio.

En el ciclo siguiente, `restore_dirty` repone el fondo capturado de la moneda y `draw_dirty` deja de dibujarla. Mantenga el puntero en la lista hasta completar ese ciclo; retirarlo antes impediría restaurar su fondo.

Reserve las filas 1 y 2 del mapa como interior negro del HUD y mantenga los bordes laterales fuera del texto. El ejemplo coloca puntuación en `x=8,y=8` y ayuda/estado en `x=32,y=8`. Como el texto con máscara no borra el glifo anterior, cada cambio normal restaura solo la columna de tile que contiene el dígito —dos celdas por sus nueve scanlines— y dibuja un único carácter. La victoria restaura únicamente las 18 celdas del tramo de estado. El HUD se actualiza después de `draw_dirty`, cuando el jugador ya vuelve a estar visible.



Captura 4. El recorrido automatizado actual recogió las cuatro monedas, restauró sus celdas de 4x8 bytes, actualizó el HUD y publicó la victoria en CPC 6128.

9 Construir y ejecutar

9.1 Construcción aislada

```
python3 tools/build_examples.py mi_plataformas
```

El constructor valida las fuentes y módulos, genera la configuración de vídeo, prepara `.build/mi_plataformas` y ejecuta su Makefile CPCTelera. Los artefactos principales quedan en:

```
.build/mi_plataformas/obj/mi_plataformas.bin
.build/mi_plataformas/mi_plataformas.cdt
.build/mi_plataformas/mi_plataformas.dsk
.build/mi_plataformas/selected-modules.json
```

El JSON generado permite auditar qué fuentes, módulos, medios y ruta de CPCTelera entraron realmente en la construcción.

Construccion reproducible

```
$ python3 tools/build_examples.py platformer
== building platformer ==
Compilacion terminada sin errores.

$ stat -c '%n  %s bytes' .build/platformer/obj/platformer.bin \
    .build/platformer/platformer.cdt
.build/platformer/obj/platformer.bin  9067 bytes
.build/platformer/platformer.cdt  9913 bytes

$ test -f .build/platformer/selected-modules.json && echo manifiesto-ok
manifiesto-ok
```

Captura 5. Construcción real de `platformer`; los tamaños se leen de los artefactos producidos y pueden cambiar cuando cambie el código.

9.2 Cargar el resultado

Para CDT, abra `.build/mi_plataformas/mi_plataformas.cdt` en un CPC o emulador y use los comandos normales de cinta de su entorno. Una construcción aislada también produce `mi_plataformas.dsk`; al ejecutar el flujo completo del repositorio, la política pública reúne los BIN en el DSK común descrito a continuación.

La compilación y los smokes automatizados no sustituyen una prueba visual en hardware real. Consulte `docs/validation.md` para conocer qué se ha verificado y qué sigue sin verificarse.

10 Integrar el proyecto en el DSK común

La construcción aislada no añade automáticamente un proyecto nuevo a `make examples`. Para convertirlo en ejemplo mantenido por el repositorio:

1. Añada `mi_plataformas` a `EXAMPLES` en el Makefile raíz.
2. Añádalo a la lista predeterminada de `tools/build_test_disk.py`.
3. Si debe distribuirse en releases, añádalo a la lista correspondiente de `tools/package_release.py`.
4. Documente el ejemplo en el manual y añada pruebas host/smoke apropiadas.
5. Ejecute `make examples`.

El DSK público de pruebas queda en `.build/tests/GlasnostFWK-tests.dsk` y su catálogo reproducible en `.build/tests/catalog.json`. El nombre visible del BIN procede de `amsdos_name`; no puede colisionar con otra entrada.

```
make examples
python3 -m json.tool .build/tests/catalog.json | less
```

11 Pruebas y diagnóstico

11.1 Lista de comprobación jugable

- El jugador aparece sobre una superficie, sin caer dentro del tile.
- Izquierda y derecha paran en paredes y límites del mundo.
- El salto solo se inicia desde apoyo válido.
- Las plataformas de una dirección dejan pasar al subir y sostienen al bajar.
- El fondo se restaura sin estelas al moverse.
- Un objeto recogido no reaparece y su celda queda limpia.
- El cambio de puntuación no conserva partes del dígito anterior.
- La condición de victoria puede observarse en RAM y en pantalla.

11.2 Fallos habituales

Síntoma	Comprobación concreta
El enlazador no encuentra una rutina.	Confirme que su `.s` figura en `modules` y que el símbolo está declarado con `.globl`.
El jugador se desplaza el doble.	Revise si X se introdujo como píxeles; sprites y tiles usan bytes Mode 0.
Atraviesa paredes o suelo.	Llame al resolver justo después de la física y mantenga la velocidad por debajo del tamaño del tile por frame.
No puede saltar.	Compruebe el bit `FWK_PLATFORM_TILE_GROUNDED`, la máscara de acción y que usa `fwk_input_pressed`.
Quedan estelas.	Marque `DIRTY` al mover/cambiar/ocultar, complete `restore_dirty` y `draw_dirty`, y reserve al menos `width*height` bytes por buffer.
El renderer devuelve carry.	Revise estado de fase, punteros duplicados, flags, spans, capacidades y solapamiento entre buffers activos.
El mapa no se dibuja.	Inspeccione carry tras `fwk_tilemap_render_view`, dimensiones, cámara, ids y `tile_count`.
El BIN no aparece en el DSK común.	La construcción aislada no basta: sincronice las listas de integración y vuelva a ejecutar `make examples`.

11.3 Validación proporcionada por el repositorio

```
python3 -m unittest discover -s tests -v
python3 tools/build_examples.py platformer
python3 tests/emulator_smoke.py platformer --model 6128
python3 tests/emulator_smoke.py platformer --model 464
```

La última pareja depende de que el entorno tenga configurado el emulador empleado por el smoke. Un resultado correcto confirma el recorrido automatizado que codifique la prueba; no demuestra todas las rutas jugables posibles.

12 Regenerar guía, capturas y PDF

Las tres capturas de terminal no están editadas a mano: el script lee estructura y manifiesto, construye `platformer` y consulta los tamaños resultantes. Regénerelas desde la raíz:

```
make platform-guide PYTHON=.venv-docs/bin/python
```

Para regenerar imágenes sin recompilar, solo si ya existen artefactos actuales:

```
.venv-docs/bin/python docs/images/platformer/generate_captures.py --skip-build
.venv-docs/bin/python tools/build_manual.py \
  --html docs/platform-game-guide.html \
  --pdf docs/platform-game-guide.pdf \
  --checksum docs/platform-game-guide.sha256
```

Las dos capturas jugables se regeneran desde el endpoint PNG del emulador. El smoke las toma al terminar la inicialización y al alcanzar la victoria:

```
python3 tests/emulator_smoke.py platformer --model 6128 \
  --screenshot-dir docs/images/platformer
```

> **Evidencia reproducible.** Una captura solo se escribe después de que el smoke haya observado el estado correspondiente. El comando requiere que el wrapper configurado en `--emulator` pueda iniciar AmSpiriT-Lite y su servidor local.

Referencias del repositorio

- `docs/manual.html`: API general y contratos.
- `docs/physics-profiles.md`: estado y configuración del perfil compacto.
- `docs/tilemap-collision.md`: consulta y resolución contra el mapa.
- `docs/tilemap-render.md`: descriptores y render completo/parcial.
- `docs/sprites.md`: descriptor, clipping, captura y restauración.
- `docs/render-order.md`: prefijo de entidad, ciclo dirty, solapamientos y orden.
- `examples/platformer`: aplicación completa sobre la que se valida esta guía.